

Lab 6

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [2]: DATA_PATH= "Ecommerce Customers.csv"
df = pd.read_csv(DATA_PATH)
df.head(10)
```

Out[2]:

	Email	Address	Avatar	Avg. Session Length	
0	mstephenson@fernandez.com	835 Frank Tunnel\nWrightmouth, MI 82180-9605	Violet	34.497268	12
1	hduke@hotmail.com	4547 Archer Common\nDiazchester, CA 06566-8576	DarkGreen	31.926272	11
2	pallen@yahoo.com	24645 Valerie Unions Suite 582\nCobbborough, D...	Bisque	33.000915	11
3	riverarebecca@gmail.com	1414 David Throughway\nPort Jason, OH 22070-1220	SaddleBrown	34.305557	13
4	mstephens@davidson-herman.com	14023 Rodriguez Passage\nPort Jacobville, PR 3...	MediumAquaMarine	33.330673	12
5	alvareznancy@lucas.biz	645 Martha Park Apt. 611\nJeffreychester, MN 6...	FloralWhite	33.871038	12
6	katherine20@yahoo.com	68388 Reyes Lights Suite 692\nJosephbury, WV 9...	DarkSlateBlue	32.021596	11
7	awatkins@yahoo.com	Unit 6538 Box 8980\nDPO AP 09026- 4941	Aqua	32.739143	12
8	vchurch@walter-martinez.com	860 Lee Key\nWest Debra, SD 97450-0495	Salmon	33.987773	13
9	bonnie69@lin.biz	PSC 2734, Box 5255\nAPO AA 98456- 7482	Brown	31.936549	11



```
In [3]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Email                  500 non-null   str    
1   Address                 500 non-null   str    
2   Avatar                 500 non-null   str    
3   Avg. Session Length    500 non-null   float64
4   Time on App             500 non-null   float64
5   Time on Website         500 non-null   float64
6   Length of Membership    500 non-null   float64
7   Yearly Amount Spent     500 non-null   float64
dtypes: float64(5), str(3)
memory usage: 69.3 KB
```

```
In [4]: df.describe()
```

```
Out[4]:
```

	Avg. Session Length	Time on App	Time on Website	Length of Membership	Yearly Amount Spent
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	33.053194	12.052488	37.060445	3.533462	499.314038
std	0.992563	0.994216	1.010489	0.999278	79.314782
min	29.532429	8.508152	33.913847	0.269901	256.670582
25%	32.341822	11.388153	36.349257	2.930450	445.038277
50%	33.082008	11.983231	37.069367	3.533975	498.887875
75%	33.711985	12.753850	37.716432	4.126502	549.313828
max	36.139662	15.126994	40.005182	6.922689	765.518462

```
In [5]: df.columns
```

```
Out[5]: Index(['Email', 'Address', 'Avatar', 'Avg. Session Length', 'Time on App',
              'Time on Website', 'Length of Membership', 'Yearly Amount Spent'],
              dtype='str')
```

```
In [7]: #Quick dataset checks
print("Shape:", df.shape)
print("\nMissing values per column:")
display(df.isna().sum().to_frame("missing_count").T)

print("\nDuplicate rows:", df.duplicated().sum())
```

```
Shape: (500, 8)
```

```
Missing values per column:
```

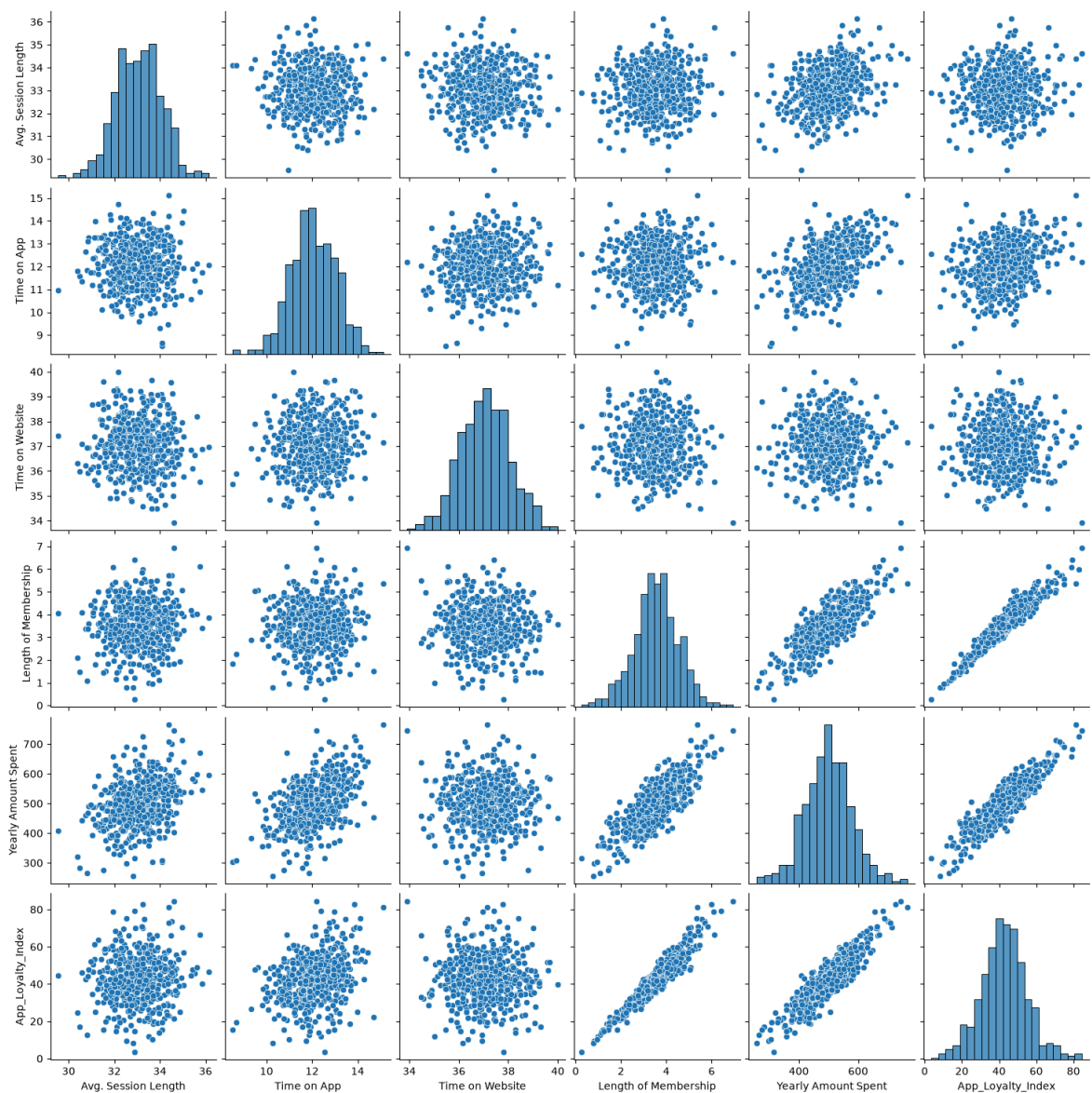
	Email	Address	Avatar	Avg. Session Length	Time on App	Time on Website	Length of Membership	Yearly Amount Spent
missing_count	0	0	0	0	0	0	0	0

Duplicate rows: 0

```
In [8]: #Feature engineering
df['Email_Provider'] = df['Email'].apply(lambda x: x.split('@')[1])
df['State'] = df['Address'].apply(lambda x: x.split()[-2])
df['App_Loyalty_Index'] = df['Time on App'] * df['Length of Membership']
```

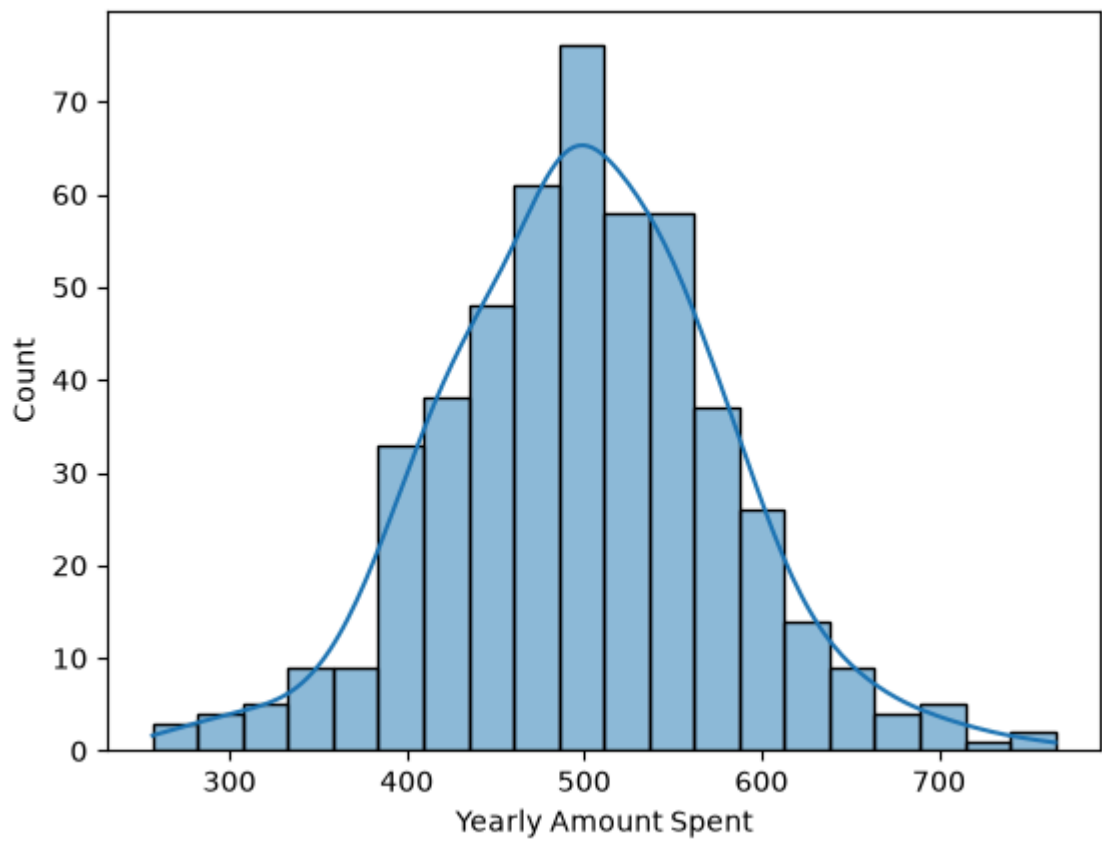
```
In [9]: #EDA
sns.pairplot(df)
```

Out[9]: <seaborn.axisgrid.PairGrid at 0x1c976cd52b0>

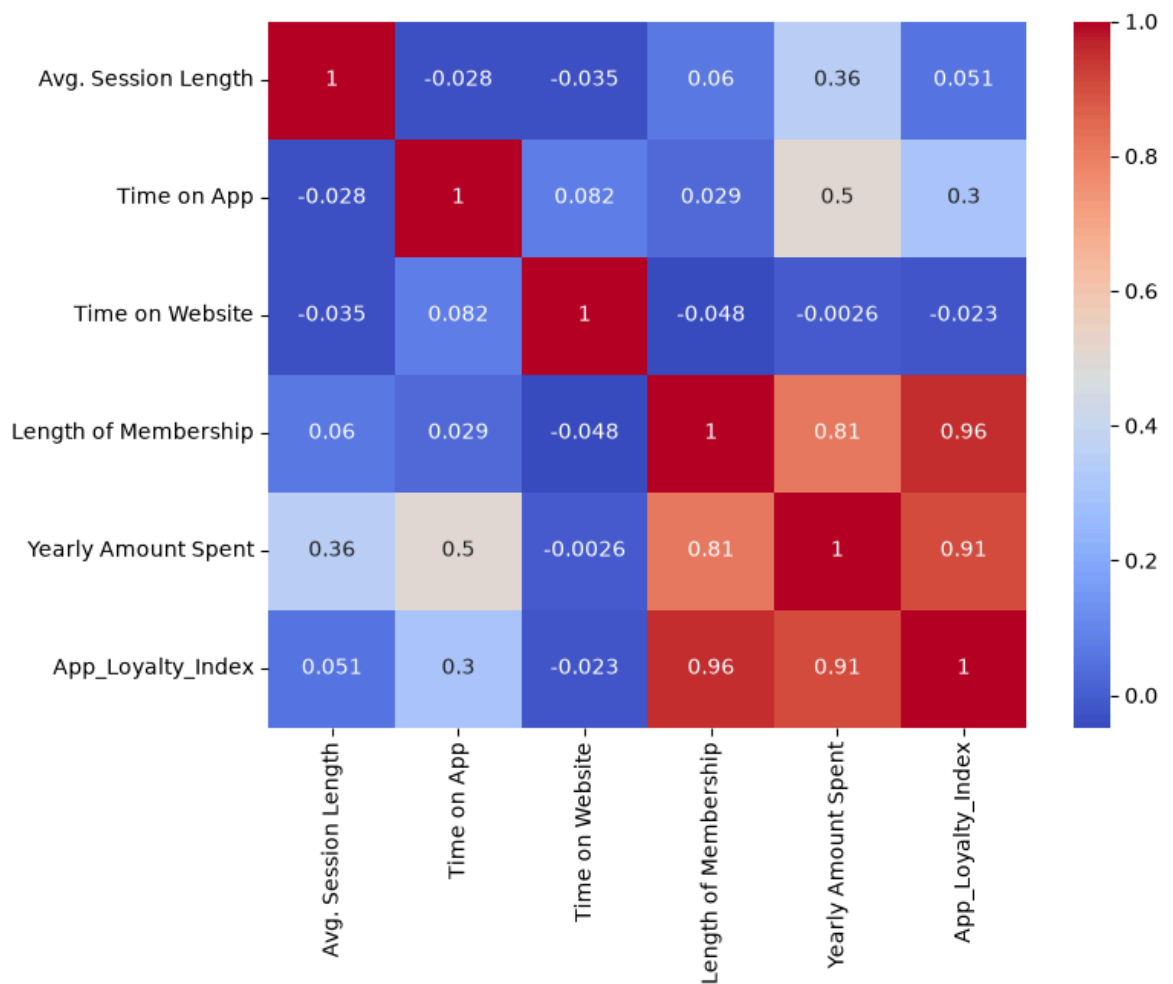


```
In [10]: sns.histplot(df['Yearly Amount Spent'], kde=True)
```

Out[10]: <Axes: xlabel='Yearly Amount Spent', ylabel='Count'>



```
In [11]: plt.figure(figsize=(8, 6))
sns.heatmap(df.select_dtypes(include='number').corr(), annot=True, cmap='coolwarm')
plt.show()
```



```
In [14]: #Prepare the data for modeling
X = df[['Avg. Session Length', 'Time on App', 'Time on Website', 'Length of Memb
y = df['Yearly Amount Spent']
```

```
In [15]: #Train Test Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_
```

```
In [16]: #Creating and Training the Model
from sklearn.linear_model import LinearRegression
lm = LinearRegression()
lm.fit(X_train, y_train)
```

```
Out[16]: ▼ LinearRegression ⓘ ?
          ▶ Parameters
          ▶ Fitted attributes
```

```
In [18]: #Model Evaluation

# print the intercept
print("Intercept:", lm.intercept_)
```

Intercept: -1045.1152168245746

```
In [19]: #print Coefficients
coeff_df = pd.DataFrame(lm.coef_, X.columns, columns=['Coefficient'])
display(coeff_df)
```

	Coefficient
Avg. Session Length	25.691540
Time on App	37.892600
Time on Website	0.560581
Length of Membership	61.648594

Coefficient Interpretation

Length of Membership (61.65): The strongest driver of sales. Every additional year of membership increases yearly spend by ~\$61.65.

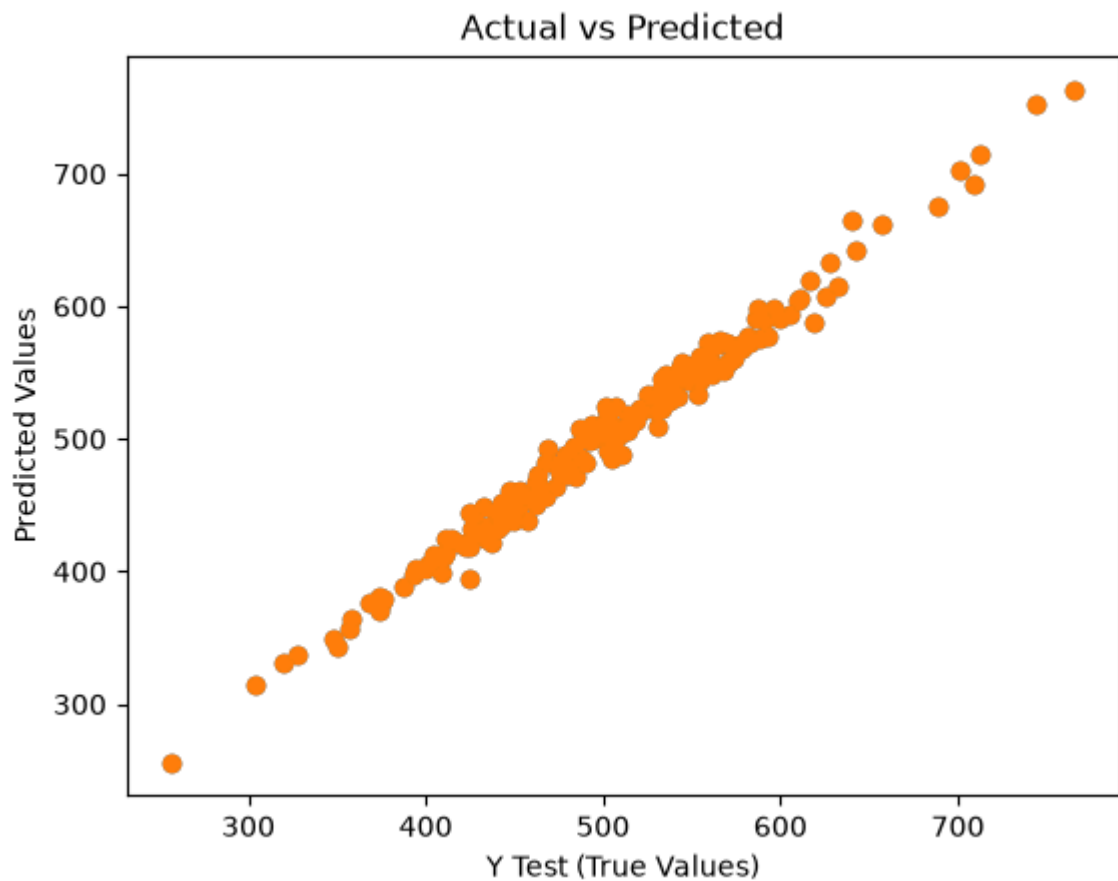
Time on App (37.89): Highly impactful. Every extra minute spent on the app increases yearly spend by ~\$37.89.

Avg. Session Length (25.69): Moderate impact on revenue.

Time on Website (0.56): The weakest factor. Time spent on the website has almost zero impact on sales.

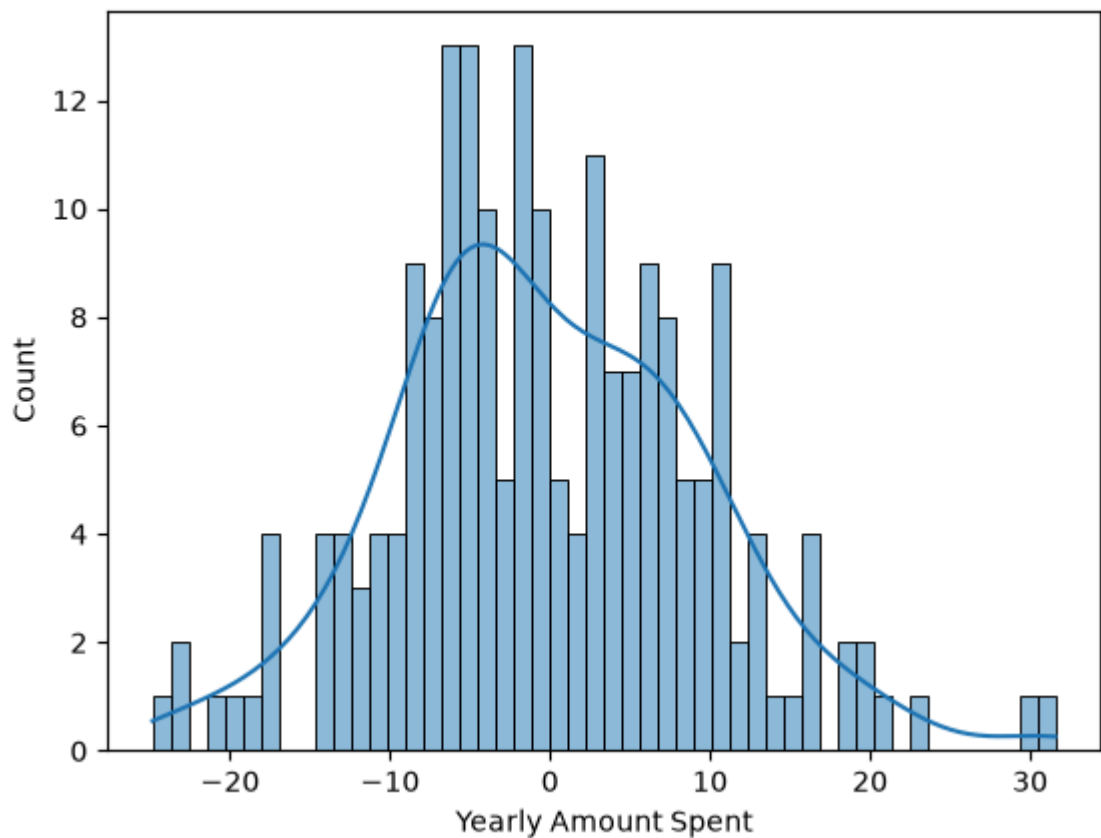
```
In [20]: #Predictions from our Model  
predictions = lm.predict(X_test)
```

```
In [21]: plt.scatter(y_test, predictions)  
plt.scatter(y_test, predictions)  
plt.xlabel('Y Test (True Values)')  
plt.ylabel('Predicted Values')  
plt.title('Actual vs Predicted')  
plt.show()
```



```
In [22]: sns.histplot((y_test - predictions), bins=50, kde=True)
```

```
Out[22]: <Axes: xlabel='Yearly Amount Spent', ylabel='Count'>
```



```
In [23]: from sklearn import metrics
```

```
print('MAE:', metrics.mean_absolute_error(y_test, predictions))  
print('MSE:', metrics.mean_squared_error(y_test, predictions))  
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, predictions)))
```

```
MAE: 7.742671285838742  
MSE: 93.83297800820081  
RMSE: 9.686742383701592
```

```
In [25]: print('R-squared:', metrics.r2_score(y_test, predictions))
```

```
R-squared: 0.9855061240024658
```